A D PATEL INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

Name : Gajera Het G.

En. No. : 12402080601044

Class : IT-A

Subject : Java(2020044502)

Batch : 1C8

Topic : ASSIGNMENT - 1

GITHUB LINK :

Assignment-1

1.Implement Array and String operations (Reverse, Sort, Search, Average,

Maximum) using class and objects

```
import java.util.Arrays;

class ArrayOperations {

    private int[] arr;

    public ArrayOperations(int[] arr) {

        this.arr = arr;

    }

    public void reverse() {

        int n = arr.length;

        for (int i = 0; i < n / 2; i++) {

            int temp = arr[i];

            arr[i] = arr[n - i - 1];

            arr[n - i - 1] = temp;

        }

    }

    public void sort() {

        Arrays.sort(arr);

    }

    public int search(int key) {
```

```
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}
public double average() {
    int sum = 0;
    for (int num : arr) {
        sum += num;
    }
    return (double) sum / arr.length;
}
public int maximum() {
    int max = arr[0];
    for (int num : arr) {
        if (num > max)
            max = num;
    }
    return max;
}
public void display() {
    System.out.println(Arrays.toString(arr));
}
```

```
}


class StringOperations {
    private String str;
    public StringOperations(String str) {
        this.str = str;
    }
    public String reverse() {
        return new StringBuilder(str).reverse().toString();
    }
    public String sort() {
        char[] chars = str.toCharArray();
        Arrays.sort(chars);
        return new String(chars);
    }
    public int search(char ch) {
        return str.indexOf(ch);
    }
    public char maximum() {
        char max = str.charAt(0);
        for (char ch : str.toCharArray()) {
            if (ch > max)
                max = ch;
        }
```

```
        return max;
    }
}
public class Main {
    public static void main(String[] args) {
        int[] arr = {5, 2, 9, 1, 7};
        ArrayOperations ao = new ArrayOperations(arr);
        System.out.print("Original Array: ");
        ao.display();
        ao.reverse();
        System.out.print("Reversed Array: ");
        ao.display();
        ao.sort();
        System.out.print("Sorted Array: ");
        ao.display();
        System.out.println("Search 9: Index = " + ao.search(9));
        System.out.println("Average = " + ao.average());
        System.out.println("Maximum = " + ao.maximum());
        StringOperations so = new StringOperations("hello");
        System.out.println("\nOriginal String: hello");
        System.out.println("Reversed String: " + so.reverse());
        System.out.println("Sorted String: " + so.sort());
        System.out.println("Search 'e': Index = " + so.search('e'));
        System.out.println("Maximum Character = " + so.maximum());
```

```
    }

}
```

OUTPUT : -

```
Original Array: [5, 2, 9, 1, 7]
Reversed Array: [7, 1, 9, 2, 5]
Sorted Array: [1, 2, 5, 7, 9]
Search 9: Index = 4
Average = 4.8
Maximum = 9

Original String: hello
Reversed String: olleh
Sorted String: ehllo
Search 'e': Index = 1
Maximum Character = o

C:\Users\newma\OneDrive\Desktop\JAVA>
```

2.Develop Matrix class with constructors, transpose and multiplication

```
class Matrix {
    private int rows;
    private int cols;
    private int[][] data;
    public Matrix() {
        this.rows = 2;
        this.cols = 2;
        data = new int[rows][cols];
    }
    public Matrix(int rows, int cols) {
        this.rows = rows;
        this.cols = cols;
        data = new int[rows][cols];
    }
    public Matrix(int[][] input) {
        this.rows = input.length;
        this.cols = input[0].length;
        data = new int[rows][cols];

        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
```

```
            data[i][j] = input[i][j];
    }
    public void setValue(int i, int j, int value) {
        data[i][j] = value;
    }
    public int getValue(int i, int j) {
        return data[i][j];
    }
    public void display() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(data[i][j] + " ");
            }
            System.out.println();
        }
    }

    public Matrix transpose() {
        Matrix result = new Matrix(cols, rows);
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                result.data[j][i] = data[i][j];
        return result;
    }
```

```
    public Matrix multiply(Matrix other) {
        if (this.cols != other.rows) {
            throw new IllegalArgumentException("Matrix multiplication not
possible");
        }
        Matrix result = new Matrix(this.rows, other.cols);
        for (int i = 0; i < this.rows; i++)
            for (int j = 0; j < other.cols; j++)
                for (int k = 0; k < this.cols; k++)
                    result.data[i][j] += this.data[i][k] * other.data[k][j];
        return result;
    }
}
public class Main {
    public static void main(String[] args) {
        int[][] a = { {1, 2, 3},{4, 5, 6}};
        int[][] b = {{7, 8},{9, 10},{11, 12}};
        Matrix m1 = new Matrix(a);
        Matrix m2 = new Matrix(b);
        System.out.println("Matrix A:");
        m1.display();
        System.out.println("\nMatrix B:");
        m2.display();
```

```
        Matrix t = m1.transpose();
        System.out.println("\nTranspose of A:");
        t.display();
        Matrix product = m1.multiply(m2);
        System.out.println("\nA x B:");
        product.display();
    }
}
```

OUTPUT:-

```
Matrix A:
1 2 3
4 5 6

Matrix B:
7 8
9 10
11 12

Transpose of A:
1 4
2 5
3 6

A x B:
58 64
139 154

C:\Users\newma\OneDrive\Desktop\JAVA>
```

3.Write a program demonstrating Wrapper classes and String vs String Buffer

```
public class Main {
  public static void main(String[] args) {
    int primitiveInt = 10;
    Integer wrapperInt = primitiveInt;
    int unboxedInt = wrapperInt;
    System.out.println("Primitive int: " + primitiveInt);
    System.out.println("Wrapper Integer: " + wrapperInt);
    System.out.println("Unboxed int: " + unboxedInt);
    String strNum = "123";
    int parsedInt = Integer.parseInt(strNum);
    System.out.println("Parsed int from String: " + parsedInt);
    Integer obj1 = 100;
    Integer obj2 = 100;
    System.out.println("Wrapper comparison (==): " + (obj1 == obj2));
    System.out.println("Wrapper comparison (equals): " + obj1.equals(obj2));

    String s1 = "Hello";
    s1.concat(" World");
    System.out.println("String after concat: " + s1);
    String s2 = s1.concat(" World");
    System.out.println("New String: " + s2);
```

```
        StringBuffer sb = new StringBuffer("Hello");
        sb.append(" World");
        System.out.println("StringBuffer after append: " + sb);
        System.out.println("StringBuffer capacity: " + sb.capacity());
        sb.ensureCapacity(50);
        System.out.println("Updated capacity: " + sb.capacity());
    }
}
```

OUTPUT:-

```
Primitive int: 10
Wrapper Integer: 10
Unboxed int: 10
Parsed int from String: 123
Wrapper comparison (==): true
Wrapper comparison (equals): true
String after concat: Hello
New String: Hello World
StringBuffer after append: Hello World
StringBuffer capacity: 21
Updated capacity: 50

C:\Users\newma\OneDrive\Desktop\JAVA>
```

4.Implement BankAccount class with deposit, withdraw and balance inquiry

```
class BankAccount {
    private String accountHolder;
    private int accountNumber;
    private double balance;
    public BankAccount(String name, int accNo, double initialBalance) {
        this.accountHolder = name;
        this.accountNumber = accNo;
        this.balance = initialBalance;
    }
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount");
        }
    }
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
```

```
        } else {
            System.out.println("Insufficient balance or invalid amount");
        }
    }
    public void displayBalance() {
        System.out.println("Account Holder: " + accountHolder);
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Balance: " + balance);
    }
}
public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("John Doe", 12345, 1000.0);
        acc.displayBalance();
        acc.deposit(500);
        acc.withdraw(300);
        acc.withdraw(1500);
        acc.displayBalance();
    }
}
```

OUTPUT:-

```
Account Holder: John Doe
Account Number: 12345
Balance: 1000.0
Deposited: 500.0
Withdrawn: 300.0
Insufficient balance or invalid amount
Account Holder: John Doe
Account Number: 12345
Balance: 1200.0

C:\Users\newma\OneDrive\Desktop\JAVA>
```

5.Develop inheritance-based Cricket–Match system using command line arguments

```
import java.util.Scanner;
class Cricket {
    String team1, team2;
    Cricket(String t1, String t2) {
        team1 = t1;
        team2 = t2;
    }
    void showTeams() {
        System.out.println("Match Between: " + team1 + " vs " + team2);
    }
}
class Match extends Cricket {
    int score1, score2;
    Match(String t1, String t2, int s1, int s2) {
        super(t1, t2);
        score1 = s1;
        score2 = s2;
    }
    void result() {
        if (score1 > score2)
            System.out.println(team1 + " wins!");
```

```
        else if (score2 > score1)
            System.out.println(team2 + " wins!");
        else
            System.out.println("Match Draw!");
    }
    void display() {
        showTeams();
        System.out.println(team1 + " Score: " + score1);
        System.out.println(team2 + " Score: " + score2);
        result();
    }
}
class Program5 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Team 1: ");
        String t1 = sc.nextLine();
        System.out.print("Enter Team 2: ");
        String t2 = sc.nextLine();
        System.out.print("Enter Score of " + t1 + ": ");
        int s1 = sc.nextInt();
        System.out.print("Enter Score of " + t2 + ": ");
        int s2 = sc.nextInt();
        Match m = new Match(t1, t2, s1, s2);
```

```
        m.display();
    }
}
```

OUTPUT:-

```
Enter Team 1: RCB
Enter Team 2: GT
Enter Score of RCB: 213
Enter Score of GT: 215
Match Between: RCB vs GT
RCB Score: 213
GT Score: 215
GT wins!

C:\Users\newma\OneDrive\Desktop\JAVA>
```

6.Implement Cipher system using Abstract class and method overriding

```
abstract class Cipher {
    abstract String encrypt(String text);
    abstract String decrypt(String text);
}
class CaesarCipher extends Cipher {
    private int shift;
    CaesarCipher(int shift) {
        this.shift = shift;
    }
    String encrypt(String text) {
        String result = "";
        for (char ch : text.toCharArray()) {
            if (Character.isLetter(ch)) {
                char base = Character.isUpperCase(ch) ? 'A' : 'a';
                result += (char) ((ch - base + shift) % 26 + base);
            } else {
                result += ch;
            }
        }
        return result;
    }
```

```
    String decrypt(String text) {
        String result = "";
        for (char ch : text.toCharArray()) {
            if (Character.isLetter(ch)) {
                char base = Character.isUpperCase(ch) ? 'A' : 'a';
                result += (char) ((ch - base - shift + 26) % 26 + base);
            } else {
                result += ch;
            }
        }
        return result;
    }
}
public class Main {
    public static void main(String[] args) {

        Cipher cipher = new CaesarCipher(3);
        String original = "Hello World";
        String encrypted = cipher.encrypt(original);
        String decrypted = cipher.decrypt(encrypted);
        System.out.println("Original: " + original);
        System.out.println("Encrypted: " + encrypted);
        System.out.println("Decrypted: " + decrypted);
    }
```

}

OUTPUT:-

```
Original: Hello World
Encrypted: Khoor Zruog
Decrypted: Hello World

C:\Users\newma\OneDrive\Desktop\JAVA>
```

7.Demonstrate Inner Classes (member, local, anonymous)

```
class Outer {
    private String message = "Hello from Outer";
    class MemberInner {
        void display() {
            System.out.println("Member Inner: " + message);
        }
    }
    void showLocalClass() {
        class LocalInner {
            void display() {
                System.out.println("Local Inner: " + message);
            }
        }
        LocalInner li = new LocalInner();
        li.display();
    }
    void showAnonymousClass() {
        Runnable r = new Runnable() {
            public void run() {
                System.out.println("Anonymous Inner: " + message);
            }
        };
```

```
        r.run();
    }
}
public class Main {
    public static void main(String[] args) {
        Outer outer = new Outer();
        Outer.MemberInner mi = outer.new MemberInner();
        mi.display();
        outer.showLocalClass();
        outer.showAnonymousClass();
    }
}
```

OUTPUT:-

```
Member Inner: Hello from Outer
Local Inner: Hello from Outer
Anonymous Inner: Hello from Outer

C:\Users\newma\OneDrive\Desktop\JAVA>
```

8.Create custom exception handling for bank withdrawal scenario

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}
class BankAccount {
    private double balance;
    BankAccount(double balance) {
        this.balance = balance;
    }
    void withdraw(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException("Insufficient balance.
Available: " + balance);
        } else if (amount <= 0) {
            throw new InsufficientBalanceException("Invalid withdrawal
amount");
        } else {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        }
    }
    void displayBalance() {
```

```
        System.out.println("Current Balance: " + balance);
    }
}
public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);
        acc.displayBalance();
        try {
            acc.withdraw(500);
            acc.withdraw(700);
        } catch (InsufficientBalanceException e) {
            System.out.println("Exception: " + e.getMessage());
        }
        acc.displayBalance();
    }
}
```

OUTPUT:-

```
Current Balance: 1000.0
Withdrawn: 500.0
Exception: Insufficient balance. Available: 500.0
Current Balance: 500.0

C:\Users\newma\OneDrive\Desktop\JAVA>
```